UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN

GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

BACHELOR'S THESIS

DESIGN AND IMPLEMENTATION OF FFT ARCHITECTURES WITH NON-POWER-OF-TWO SIZES FOR 5G

VÍCTOR MANUEL BAUTISTA LOZA

JULY 2021

GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

BACHELOR'S THESIS

Title:	DESIGN AND IMPLEMENTATION OF FFT ARCHITECTURES WITH NON-POWER-OF-TWO SIZES FOR 5G
Author:	VÍCTOR MANUEL BAUTISTA LOZA
Supervisor:	MARIO GARRIDO GÁLVEZ
Ponente:	MARÍA LUISA LÓPEZ VALLEJO
Department:	DEPARTAMENTO DE INGENIERÍA ELECTRÓNICA

EVALUATION COMMITTEE

President: .....

Vocal: .....

Secretary: .....

The evaluation committee members agree to grant the grade: .....

Madrid, on ....., 2021

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR DE INGENIEROS DE TELECOMUNICACIÓN

GRADO EN INGENIERÍA DE TECNOLOGÍAS Y SERVICIOS DE TELECOMUNICACIÓN

BACHELOR'S THESIS

DESIGN AND IMPLEMENTATION OF FFT ARCHITECTURES WITH NON-POWER-OF-TWO SIZES FOR 5G

Author: VÍCTOR MANUEL BAUTISTA LOZA

Supervisor: MARIO GARRIDO GÁLVEZ

Ponente: MARÍA LUISA LÓPEZ VALLEJO

JULY 2021

A mi familia, pareja y amigos.

"If you want to find the secrets of the universe, think in terms of energy, frequency and vibration", Nikola Tesla.

Abstract

The fast Fourier transform (FFT) is a fundamental element of the 5G physical layer. The physical layer description [1] details that the size of the FFT has to be a product of powers of 2, 3 and 5.

In the literature, there are numerous FFT designs for sizes that are powers-of-two, which is a field of research that has been deeply explored, and has reached a high degree of optimization [2]. In contrast, the design of FFTs for sizes that are non-powers-of-two [3, 4] is a field of research in which less progress has been made and current designs are far from been optimized. This means that, in practice, FFTs with sizes that are powers of two are generally chosen, avoiding non-power-of-two ones.

The aim of this TFG is to go further into the design of FFT architectures for sizes that are not powers-of-two, with the aim to find efficient solutions for all FFT sizes considered in 5G.

The TFG will focus on serial FFT architectures, which process one sample per clock cycle. These architectures compute the FFT by using butterflies, which calculate additions and subtractions, and rotators, which compute rotations of data in the complex number plane. Currently, the butterflies that are used in architectures for FFTs that are not powers of two have low utilization. Thus, the aim of this TFG is to design butterflies with higher utilization by reducing the number of hardware components in the butterfly and, simultaneously, use them for a larger number of clock cycles, which results in a reduction of area and power consumption.

The butterflies that have been designed in this TFG have been implemented in VHDL to verify their functionality and to obtain experimental results. These experimental results have been compared with other previous architectures.

Key Words

FPGA, VHDL, DFT, FFT, butterfly, SDC, SDF, pipeline, LUT, multiplexer, register, slices, latency, rotator.

Resumen

La transformada rápida de Fourier (FFT) es un elemento fundamental de la capa física de 5G. En la descripción de la capa física [1] se detalla que el tamaño de la FFT ha de ser un producto de potencias de 2, 3 y 5.

En la literatura existen numerosos diseños de la FFT para tamaños que son potencias de 2, siendo este un campo de investigación que se ha explorado en profundidad, llegando a un alto grado de optimización [2]. Por el contrario, el diseño de FFTs de tamaños que no son potencias de dos [3, 4] es un campo de investigación en el que no se ha avanzado tanto y los diseños actuales están lejos de estar optimizados. Ello hace que en la práctica se suela recurrir a tamaños de la FFT que son potencias de dos, descartando los tamaños que no lo son.

El objetivo de este TFG es ahondar en el diseño de arquitecturas FFT para tamaños que no son potencias de dos, lo cual está encaminado a buscar soluciones eficientes para todos los tamaños de la FFT considerados en 5G.

El TFG se va a centrar en arquitecturas serie de cálculo de la FFT, las cuales procesan una muestra por ciclo de reloj. Estas arquitecturas calculan la FFT por medio de mariposas o $butterflies$, que realizan sumas y restas, y rotadores, que calculan rotaciones de los datos en el plano de los números complejos. Actualmente, la utilización de las mariposas en arquitecturas para FFTs que no son potencias de dos es baja. Así, el objetivo de este TFG es diseñar mariposas con una mayor utilización, por medio de la reducción del número de componentes hardware en la mariposa y, simultáneamente, el uso de los mismos durante una mayor cantidad de ciclos de reloj, dando lugar a una reducción del área y del consumo de potencias.

Las mariposas diseñadas en este TFG han sido implementadas en VHDL para verificar su funcionamiento y obtener resultados experimentales. Estos resultados experimentales han sido comparados con los de arquitecturas previas.

Palabras clave

FPGA, VHDL, DFT, FFT, mariposa, SDC, SDF, pipeline, LUT, multiplexor, registro, slices, latencia, rotadores.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

The discrete Fourier transform (DFT) is a mathematical function that works as a link between the time and the frequency domain. This operation allows to represent the spectral information of a sequence of discrete samples. The FFT is an essential tool used in discrete signal processing to decompose an input signal into a sum of sinusoids that represent each frequency.

The fast Fourier transform (FFT) is an algorithm that allows to compute the DFT with a lower amount of operations than computing the DFT by itself. It was proposed by Cooley-Tukey in 1965 [5] and it has become one of the most important algorithms ever since.

This efficient algorithm is not only used in the field of signal processing but also it is commonly used for calculating numerically differential equations. For instance, in particle physics in order to calculate the Poisson equation [6] and in geology to resolve heat equations [7].

Serial FFT architectures process one sample per clock cycle. These architectures compute the FFT by using butterflies, which calculate additions and subtractions, and rotators, which compute rotations of data in the complex number plane. Currently, the butterflies that are used in architectures for FFTs that are not powers of two have low utilization. Thus, the aim of this TFG is to design butterflies with higher utilization by reducing the number of hardware components in the butterfly and, simultaneously, use them for a larger number of clock cycles, which results in a reduction of area and power consumption.

The TFG is organized as follows. Chapter 2 presents the state-of-the-art. It begins by introducing the relation that exists between non-power-of-two sizes of FFTs and new communication technologies. Then, the chapter continues describing the butterflies structures and their associated flow graphs. Chapter 3 presents the proposed butterflies and the experimental results that allow to compare the proposed circuits with parallel ones. Finally, Chapter 4 summarizes the main conclusion of

this TFG and discusses future work.

Chapter 2

Background

An N-point discrete Fourier transform (DFT) of a signal $x[n]$ is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{nk}, k = 0, 1, ..., N-1, \tag{2.1}$$

where k and n represent the frequency and the discrete time unit, respectively. The term $W_N^{nk} = e^{-j\frac{2\pi}{N}nk}$ is called twiddle factor and represents a rotation in the complex plane. The calculation of the DFT requires a lot of hardware resources by itself, but the Fourier transform (FFT) reduces the number of operations. The DFT has a complexity of $O(N^2)$, while the FFT reduces it to $O(N \cdot \log N)$ (operator $O(\cdot)$ represent the order of the number of operations).

2.1 FFTs in 5G and beyond

New communications generations use the FFT algorithm to calculate code-words during modulation and demodulation transactions. Physical layer description for 5G [1] establishes the size N of FFTs as

$$N = 2^{\alpha_2} \cdot 3^{\alpha_3} \cdot 5^{\alpha_5}, \tag{2.2}$$

where α_2, α_3, $\alpha_5 \in \mathbb{Z}^+$. This means that the door for developing non-power-of-two (NP2) FFTs is opened. Nevertheless, in practice, these new architectures are far from reaching a high degree of optimization [3, 4], so hardware designers prefer to accommodate their circuits to power-of-two (P2) ones.

For next generation technologies such as 6G [8], this issue will persist unless new optimal architectures for NP2 FFTs are designed. This thesis addresses the designing of optimized butterflies for radices 2, 3, 4 and 5, for a later combination of them in order to form NP2 FFTs with a high grade of optimization.

2.2 The FFT

The most readable way to represent an FFT algorithm is to use a flow graph. Flow graphs are drawings that help to understand the sequence of operations that generates outputs from input values. After every arithmetic operation there is a new node that can be used later as an input for others arithmetic operations. P2 FFTs graphs consist on m stages calculated as $m = \log_2 N$. Each stage $s \in \{1, 2, ...N\}$ consists on a set of butterflies and rotations.

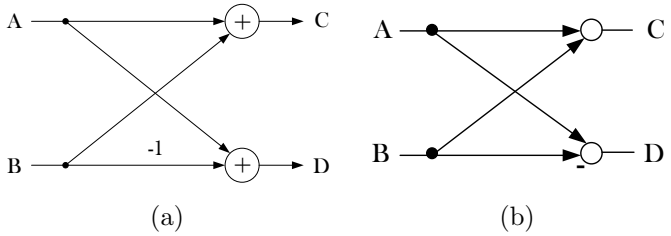

Figure 2.1: Two different representations of generic operations in a butterfly. (a) Mathematical representation. (b) Simplified representation.

Figure 2.1 represents a generic structure of a butterfly, where inputs are A and B, and outputs are C and D. It just consist of two additions where the lower branch is multiplied by -1 and it forms the basic element of FFT flow graphs. In this chapter, butterflies of radix 2, 3, 4 and 5 will be described. Radix-2 FFT means that the algorithm considers butterflies of size 2. These radix-ρ architectures will be used for calculating N-point FFTs. Figure 2.2 shows an example of an 8-point FFT using radix-2 butterflies among their decomposed stages according to decimation in frequency (DIF) [9]. $N = 8$ points implies $m = 3$ stages. Rotations appears after

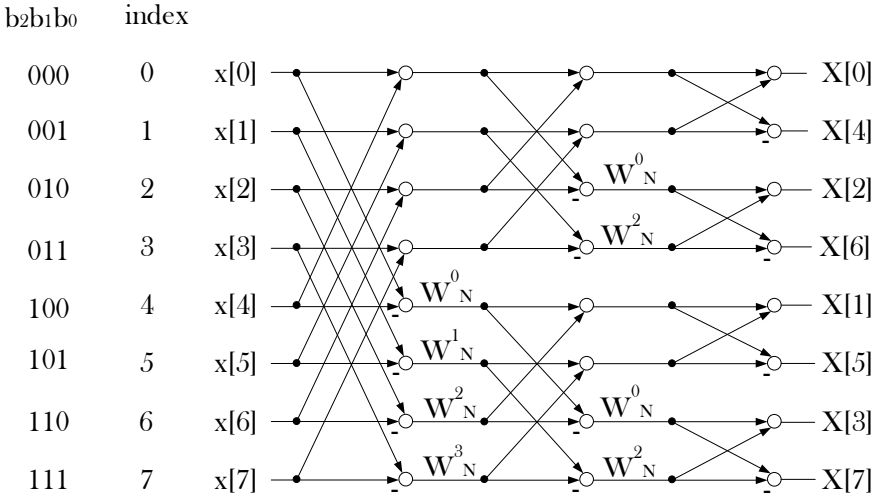

Figure 2.2: Flow graph of a 8-points radix-2 FFT

butterflies in order to multiply the node by its complex-valued twiddle factor. Any

other P2 FFT ends up in the generic radix-2 butterfly of Figure 2.1.

However, there is an additional issue related with how data is managed. By assuming that data arrives at FFT stages at a rate of one datum per clock cycle and from top to bottom of the flow graph, Figure 2.2 receives discrete input values in natural order, but provides the outputs in a scrambled order. The algorithm establishes how pairs of discrete values must be operated in the butterflies. Mathematically, if the input sequence $\{x[0], x[1], x[2], ..., x[N-1]\}$ is codified with 3 bits like $\{(000), (001), (010), ..., (111)\}$, operations are distributed so that at stage s the data pairs selected differ only in the bit b_{n-s}. For example, in the first stage, pairs of data are selected as $\{(000), (100)\}, \{(001), (101)\}, \{(010), (110)\}, \{(011), (111)\}$ so they only differ in the most significant bit b_2. The same analysis must be done for next stages for a correct calculation of butterflies, comparing b_1 in second stage and b_0 in the last one.

Serial pipelined FFT architectures with natural input order as in Figure 2.2, will need additional hardware before the first stage in order to operate $\{(000), (100)\}$. This is caused because in serial pipelined circuits only one discrete value arrives to the circuit every clock cycle, so the first value (000) must wait for (100) and that produces a queue for first $N/2$ values. An alternative to this problem is to consider that input values are provided in bit-reversed ordered. In this case, no additional hardware is needed before the first stage and the output data is provided in natural order, as is shown in Figure 2.3.

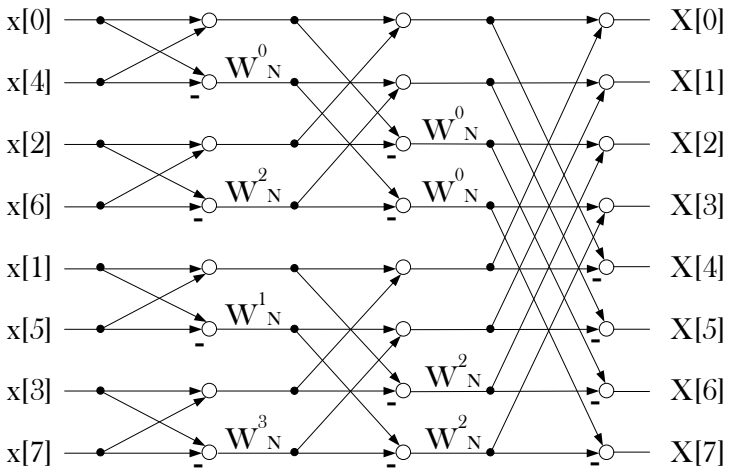

Figure 2.3: Flow graph of a 8-points radix-2 FFT

To sort out input and/or output data, bit reversal circuits are used [10]. Their work consist in sorting out data that arrive to the circuit in natural order so that access the FFT circuit as in Figure 2.3. Also, bit reversal circuits can sort out output data once the FFT has been calculated. This technique will be used in radix-4 and radix-5 butterflies.

2.2.1 Radix-2 butterfly

According to equation (2.1), a 2-point FFT is calculated as

$$X[k] = \sum_{n=0}^{1} x[n] \cdot W_N^{nk} = x[0] \cdot W_N^0 + x[1] \cdot W_N^1, \quad k = 0, 1. \tag{2.3}$$

As $k \in \{0, 1\}$, the twiddle factors are $W_N^0 = 1$ and $W_N^1 = -1$. For both values of k the outputs are:

$$X[0] = x[0] + x[1], \tag{2.4a}$$

$$X[1] = x[0] - x[1]. \tag{2.4b}$$

Equations (2.4a) and (2.4b) make it easy to draw the flow graph of a radix-2 butterfly, which is shown in Figure 2.4. It just consists in the basic butterfly representation already detailed in Figure 2.1.

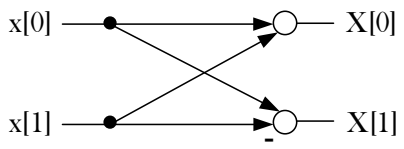

Figure 2.4: Flow graph of a radix-2 butterfly

2.2.2 Radix-3 butterfly

According to equation (2.1), a 3-point FFT is calculated as

$$X[0] = \sum_{n=0}^{2} x[n] \cdot W_3^0 = (x[0] + x[1] + x[2]) \cdot W_3^0, \tag{2.5a}$$

$$X[1] = \sum_{n=0}^{2} x[n] \cdot W_3^n = x[0] \cdot W_3^0 + x[1] \cdot W_3^1 + x[2] \cdot W_3^2, \tag{2.5b}$$

$$X[2] = \sum_{n=0}^{2} x[n] \cdot W_3^n = x[0] \cdot W_3^0 + x[1] \cdot W_3^2 + x[2] \cdot W_3^4. \tag{2.5c}$$

Table 2.1 shows the exact values for the twiddle factors in a 3-point FFT as a function of n and k.

n,k	0	1	2
0	1	1	1
1	1	$-\frac{1}{2}-j\frac{\sqrt{3}}{2}$	$-\frac{1}{2}+j\frac{\sqrt{3}}{2}$
2	1	$-\frac{1}{2}+j\frac{\sqrt{3}}{2}$	$-\frac{1}{2}-j\frac{\sqrt{3}}{2}$

Table 2.1: Values of W_3^{nk} used in the set of equations (2.5)

Equations (2.5a), (2.5b), (2.5c) can be rewritten by using Table 2.1, leading to

$$X[0] = x[0] + x[1] + x[2], \tag{2.6a}$$

$$X[1] = x[0] - \frac{1}{2}(x[1] + x[2]) - j\frac{\sqrt{3}}{2}(x[1] - x[2]), \tag{2.6b}$$

$$X[2] = x[0] - \frac{1}{2}(x[1] + x[2]) + j\frac{\sqrt{3}}{2}(x[1] - x[2]). \tag{2.6c}$$

According to equations (2.6a), (2.6b) and (2.6c), the flow graph of a radix-3 butterfly [11] can be drawn as

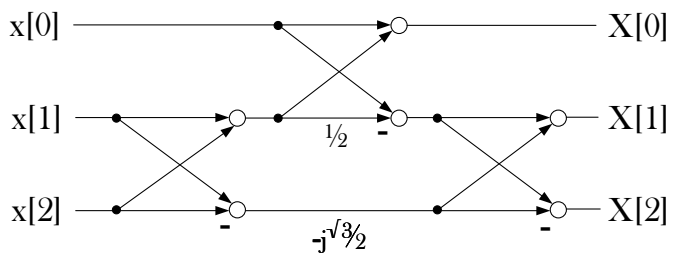

Figure 2.5: Flow graph of a radix-3 butterfly

The radix-3 butterfly consists in three stages of butterflies and one multiplier by $-j\frac{\sqrt{3}}{2}$. Multiplications by 2^m can be implemented in hardware by adding or removing bits, which does not have any hardware cost. This applies to the multiplication by $\frac{1}{2}$ in the flow graph.

2.2.3 Radix-4 butterfly

According to equation (2.1), a 4-point FFT is calculated as

$$X[0] = \sum_{n=0}^{3} x[n] \cdot W_4^0 = (x[0] + x[1] + x[2] + x[3]) \cdot W_4^0, \tag{2.7a}$$

$$X[1] = \sum_{n=0}^{3} x[n] \cdot W_4^n = x[0] \cdot W_4^0 + x[1] \cdot W_4^1 + x[2] \cdot W_4^2 + x[3] \cdot W_3^3, \tag{2.7b}$$

$$X[2] = \sum_{n=0}^{3} x[n] \cdot W_4^n = x[0] \cdot W_4^0 + x[1] \cdot W_4^2 + x[2] \cdot W_4^4 + x[3] \cdot W_4^6, \tag{2.7c}$$

$$X[3] = \sum_{n=0}^{3} x[n] \cdot W_4^n = x[0] \cdot W_4^0 + x[1] \cdot W_4^3 + x[2] \cdot W_4^6 + x[3] \cdot W_4^9. \tag{2.7d}$$

Table 2.2 shows the exact values for the twiddle factors in a 4-point FFT as a function of n and k.

n, k	0	1	2	3
0	1	1	1	1
1	1	-j	-1	j
2	1	-1	1	-1
3	1	j	-1	-j

Table 2.2: Values of W_4^{nk} used in the set of equations (2.7)

Equations (2.7a), (2.7b), (2.7c) and (2.7d) can be rewritten by using Table 2.2 as

$$X[0] = x[0] + x[2] + x[1] + x[3], \tag{2.8a}$$

$$X[1] = x[0] - x[2] - j(x[1] - x[3]), \tag{2.8b}$$

$$X[2] = x[0] + x[2] - (x[1] + x[3]), \tag{2.8c}$$

$$X[3] = x[0] - x[2] + j(x[1] - x[3]). \tag{2.8d}$$

In the set of operations (2.8) there are some terms that are reused. This results in a smaller number of operations in the flow graph. Figure 2.6 represents the flow graph of a radix-4 butterfly, which results from the set of operations (2.8).

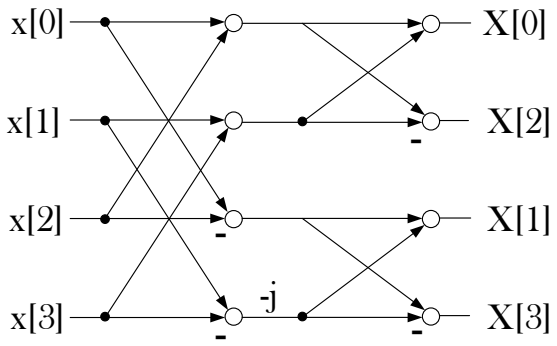

Figure 2.6: Flow graph of a radix-4 butterfly

If the outputs are required in natural order, a bit reversal circuit that sorts out output data into natural order will be required. This circuit does not need any multiplier. Likewise, when a factor is been multiplied by a pure module-one complex number such as j, it interchanges its real and imaginary part and then multiplies its new real part by -1, for which no multiplier is needed.

2.2.4 Radix-5 butterfly

A radix-5 FFT can be calculated as

$$X[0] = \sum_{n=0}^{4} x[n] \cdot W_5^0 = (x[0] + x[1] + x[2] + x[3] + x[4]) \cdot W_5^0, \tag{2.9a}$$

$$X[1] = \sum_{n=0}^{4} x[n] \cdot W_5^n = x[0] \cdot W_5^0 + x[1] \cdot W_5^1 + x[2] \cdot W_5^2 + x[3] \cdot W_5^3 + x[4] \cdot W_5^4, \tag{2.9b}$$

$$X[2] = \sum_{n=0}^{4} x[n] \cdot W_5^{2n} = x[0] \cdot W_5^0 + x[1] \cdot W_5^2 + x[2] \cdot W_5^4 + x[3] \cdot W_5^6 + x[4] \cdot W_5^8. \tag{2.9c}$$

$$X[3] = \sum_{n=0}^{4} x[n] \cdot W_5^{3n} = x[0] \cdot W_5^0 + x[1] \cdot W_5^3 + x[2] \cdot W_5^6 + x[3] \cdot W_5^9 + x[4] \cdot W_5^{12}, \tag{2.9d}$$

$$X[4] = \sum_{n=0}^{4} x[n] \cdot W_5^{4n} = x[0] \cdot W_5^0 + x[1] \cdot W_5^4 + x[2] \cdot W_5^8 + x[3] \cdot W_5^{12} + x[4] \cdot W_5^{16}. \tag{2.9e}$$

Table 2.3 shows the exact values for the twiddle factors in a 5-point FFT as a function of n and k.

n, k	0	1	2	3	4
0	1	1	1	1	
1	1	0.3090 - j0.9511	-0.8090 - j0.5878	-0.8090 + j0.5878	0.3090 + j0.9511
2	1	-0.8090 - j0.5878	0.3090 + j0.9511	0.3090 - j0.9511	-0.8090 + j0.5878
3	1	-0.8090 + j0.5878	0.3090 - j0.9511	0.3090 + j0.9511	-0.8090 - j0.5878
4	1	0.3090 + j0.9511	-0.8090 + j0.5878	-0.8090 - 0.5878	0.3090 - j0.9511

Table 2.3: Values of W_5^{nk} used in the set of equations (2.9)

Equations (2.9a), (2.9b), (2.9c), (2.9d) and (2.9e) can be rewritten by using Table 2.3 and grouping terms [12, 13] as

$$X[0] = x[0] + x[1] + x[2] + x[3] + x[4], \tag{2.10a}$$

$$\begin{aligned} X[1] = x[0] &+ K_1\left(x[1] + x[4] + x[2] + x[3]\right) + K_2\left((x[1] + x[4]) - (x[2] + x[3])\right) \\ &+ K_3(x[1] - x[4]) + K_4\left((x[1] - x[4]) + (x[2] - x[3])\right), \end{aligned} \tag{2.10b}$$

$$\begin{aligned} X[2] = x[0] &+ K_1\left(x[1] + x[4] + x[2] + x[3]\right) - K_2\left((x[1] + x[4]) - (x[2] + x[3])\right) \\ &+ K_5(x[2] - x[3]) + K_4\left((x[1] - x[4]) + (x[2] - x[3])\right), \end{aligned} \tag{2.10c}$$

$$\begin{aligned} X[3] = x[0] &+ K_1\left(x[1] + x[4] + x[2] + x[3]\right) - K_2\left((x[1] + x[4]) - (x[2] + x[3])\right) \\ &- K_5(x[2] - x[3]) - K_4\left((x[1] - x[4]) + (x[2] - x[3])\right), \end{aligned} \tag{2.10d}$$

$$\begin{aligned} X[4] = x[0] &+ K_1\left(x[1] + x[4] + x[2] + x[3]\right) + K_2\left((x[1] + x[4]) - (x[2] + x[3])\right) \\ &- K_3(x[1] - x[4]) - K_4\left((x[1] - x[4]) + (x[2] - x[3])\right), \end{aligned} \tag{2.10e}$$

where K_1, K_2, K_3, K_4 and K_5 are constants given by lineal combinations of the twiddle factors resulting of the DFT operation. Their values are

$$K_1 = \frac{1}{2}\left(\cos(\frac{2\pi}{5}) + \cos(\frac{4\pi}{5})\right) = -\frac{1}{4}, \tag{2.11a}$$

$$K_2 = \frac{1}{2}\left(\cos(\frac{2\pi}{5}) - \cos(\frac{4\pi}{5})\right) = 0.56, \tag{2.11b}$$

$$K_3 = j\left(\sin(\frac{4\pi}{5}) - \sin(\frac{2\pi}{5})\right) = -j \cdot 0.36, \tag{2.11c}$$

$$K_4 = -j\sin(\frac{4\pi}{5}) = -j \cdot 0.59, \tag{2.11d}$$

$$K_5 = j\left(\sin(\frac{4\pi}{5}) + \sin(\frac{2\pi}{5})\right) = j \cdot 1.54, \tag{2.11e}$$

Figure 2.7 represent the flow graph of a radix-5 butterfly based on the previous equations.

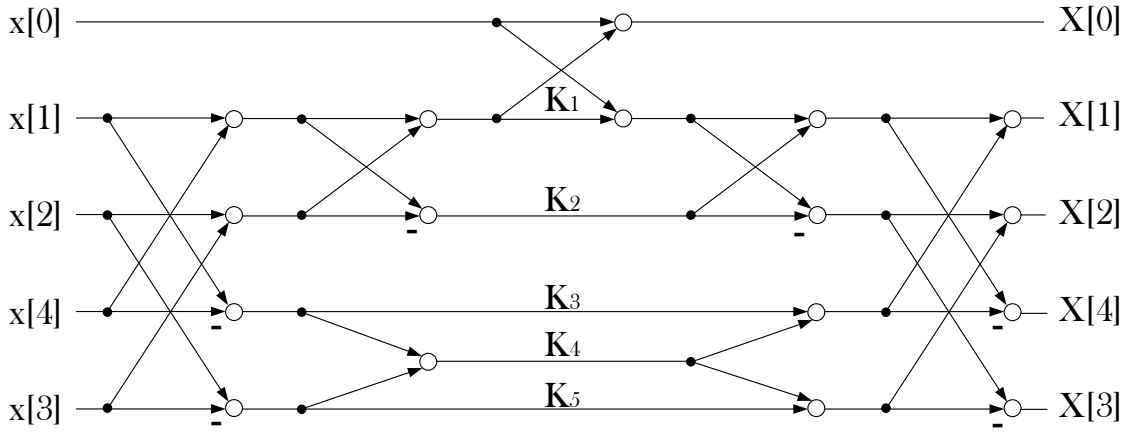

Figure 2.7: Flow graph of a radix-5 butterfly

As a result, a quite complex butterfly is composed. The flow graph shows that input data must arrive in an special order that is not bit-reversed and output data must be transformed into a natural order. Five complex multiplications and 17 complex additions must be calculated.

2.3 Permutation circuits

Flow graphs were detailed in the last section. Now, these flow graphs must be transformed into hardware circuits. The essence of the pipelined designs that are proposed in the next chapter are butterflies and permutation circuits [14]. In this section, last ones are detailed. In hardware circuits there exist data in series and in parallel, so data is visualized in two dimensions.

2.3.1 Serial-parallel permutation

Serial-parallel (SP) permutation circuits transfer data between serial and parallel dimensions. The latency of these circuits is given by the time separation of the values

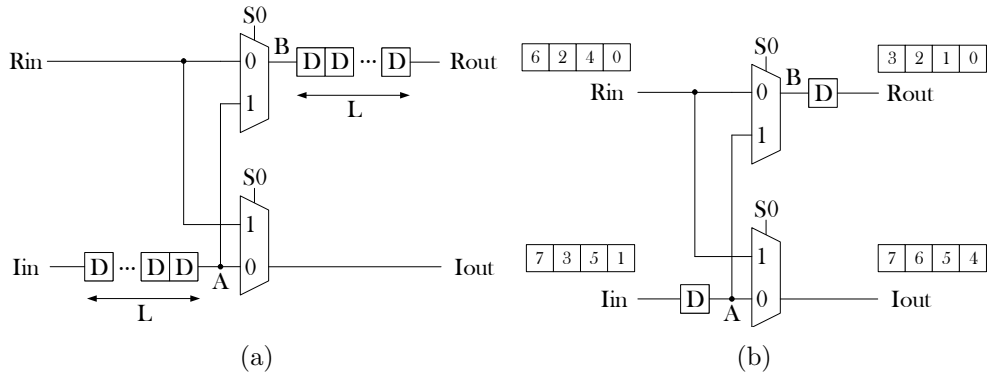

Figure 2.8: Serial-parallel permutation circuits. (a) Generic SP shuffling circuit. (b) Example of an SP circuit for length $L = 1$.

that are interchanged. Examples of SP circuits are represented in Figure 2.8.

The most basic SP circuit only costs in hardware two multiplexers and two registers. Its latency is just one clock cycle. Figure 2.8b only interchanges 1 consecutive value but the generic SP in Figure 2.8a allows to shuffle L values separated L clock cycles by using 2 multiplexers and $2L$ registers. Its latency is L.

2.3.2 Serial-serial permutation

Serial-serial (SS) permutation circuits allow the interchange of samples within the same branch.

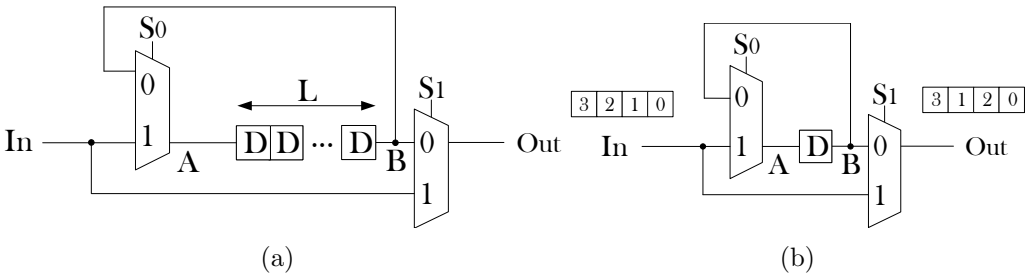

Figure 2.9: Serial-serial permutation circuits. (a) Generic SS shuffling circuit. (b) Example of an SP circuit for length $L = 1$.

Figure 2.9b represents an example of an SS shuffling circuit that interchanges two samples. The latency of this operation is one clock cycle due to the presence of only one register. Figure 2.9a represents a generic SS permutation in which L determines the distance between both samples that must be interchanged, measured in clock cycles. As a result, the cost of a generic permutation of this type is L registers and two multiplexers.

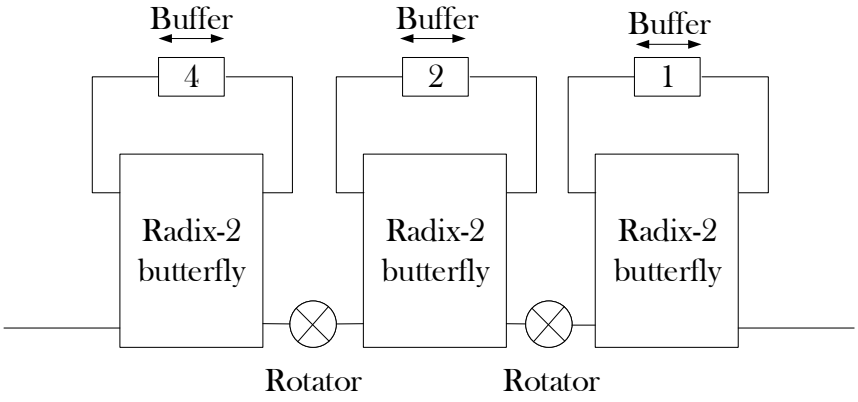

Figure 2.10: SDF architecture for 8 points FFT using radix-2 butterflies.

2.4 Butterfly utilization in SDF FFT architectures

There are several ways to compute an FFT. It can be calculated in an iterative way [15] or in a continuous data flow. The last case corresponds to pipelined architectures, which can be divided into parallel and serial ones. In this section, some types of serial architectures are introduced.

Single-path delay feedback (SDF) are serial pipelined architectures used to compute FFTs [16, 17, 18]. As every other serial pipelined architecture, they process one sample per clock cycle. SDF architectures are the most commonly used ones in order to make a pipelined circuit of FFT.

Figure 2.10 shows the implementation of Figure 2.2 by using an SDF FFT architecture. The first sample must wait for the fourth one. As a consequence, buffers are needed to make the first half of the samples wait for the last half at each stage. Once the output of a stage is ready, data go through the rotator.

There are other serial pipelined architectures such as the single-delay commutator (SDC)[19, 20, 21] or the serial commutator (SC)[22, 23]. In order to calculate a NP2 FFT nowadays, the only way is to use an SDF architecture for a serial implementation [21, 24, 25] with the only exception of [3]. The problem with SDF architectures is that the butterflies just work half of the time with 50% utilization. The goal of this TFG is to design new butterflies that work all the time. This means 100% utilization of each adder on the circuit.

Chapter 3

Proposed architectures

The main goal of this TFG is to design hardware circuits for radix-ρ butterflies, with $\rho = 2$, 3, 4 and 5, with high grade of utilization, processing one input per clock cycle. So far, existing butterflies process ρ inputs in parallel. The main feature of the proposed butterflies is that they have only one input and data are processed in series. It does not matter how long it takes for the first sample to reach the output. Once the first output frequency is calculated, continuous dataflow is expected at the output. Thus, the proposed butterflies could calculate as many DFTs as it is required with no stall cycles.

The proposed designs are serial pipelined implementations with an optimized number of resources. The circuits have two branches, one for the real part of the data and other for its imaginary part. Each circuit provides the real and imaginary part of the output frequencies at the same time on both branches. All operations are calculated in a serial manner, so the proposed architectures do not parallelize data to compute their respective operations and then serialize again the data as SDF architectures do.

The way in which the circuits have been modeled is described in section 3.1. Then, resulting designs and comparisons between the proposed implementations and the equivalent parallel circuits are presented in sections 3.2, 3.3, 3.4 and 3.5 for radices 2, 3, 4 and 5 respectively.

3.1 Methodology

In this section, the techniques used in the design of the proposed circuits are introduced, so that the reader can identify them later during the implementation. Inputs are complex-valued and circuits only can use real adders, so it is essential to describe how to manage real and imaginary parts.

From the flow graphs described in chapter 2, the minimum number of adders and multipliers are given by

$$\text{Minimum number of real adders} = \frac{\text{Total of real adds operations}}{N}, \tag{3.1}$$

$$\text{Minimum number of real multipliers} = \frac{\text{Total of multiplications}}{N}. \tag{3.2}$$

The reason for this is that there is a new input each clock cycle so that the amount of additions and multiplications used for processing N inputs must be divided by N clock cycles.

The main shuffling circuit that has been used in the designs is the parallel-parallel permutation circuit (Figure 3.1a). It consist of one serial-serial permutation circuit per branch, followed by a serial-parallel permutation circuit. An alternative implementation of this circuit is shown in Figure 3.1b. This alternative saves two multiplexers and keeps the same functionality. It is included as part of the hardware design of radix-4 and radix-5 butterflies.

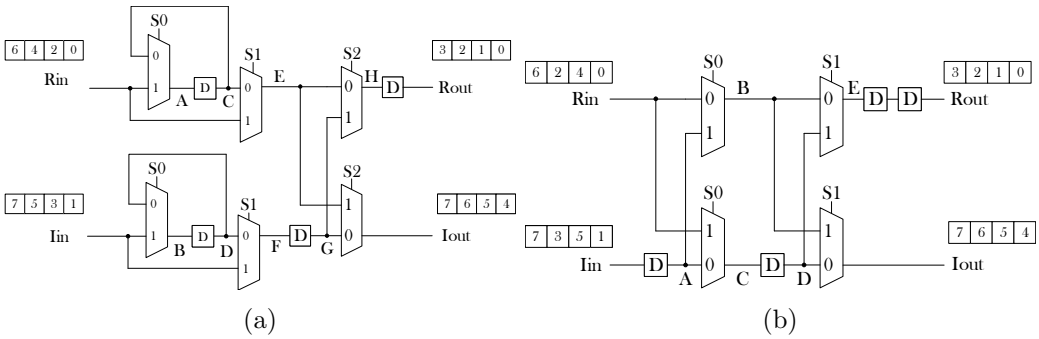

Figure 3.1: Parallel-Parallel permutation circuits. Combination of shuffling circuits (a) and the optimized circuit

Another technique used in the designs consists in bypassing some calculation. This technique is used when the output frequencies have been calculated before the last adders, which happens in radix-3 and radix-5 butterflies. Figure 3.2a and 3.2b represents the part of the circuit where this technique is applied. The real and imaginary parts of a datum can cross the a stage without been operated among them. This is possible by adding an AND logic gate between the branch and the adder. To this AND gate a control signal is connected. When it is at high level (1 logic), adders work as they are expected and when it is at low level (0 logic) the adder adds a zero. The cost of this trick is that the lower branch changes its sign so it will be common to see a multiplexer that choose between the branch that is operated or the point immediately before the adder.

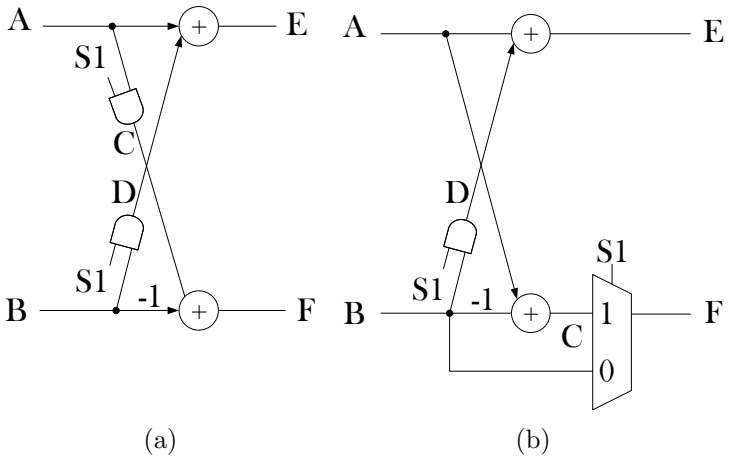

Figure 3.2: Techniques to bypass operations. (a) Lower branch output changes its sign. (b) Output keeps its sign.

3.2 Radix-2 butterfly implementation

The equations of a radix-2 butterfly are already presented in (2.4). Each input can be written as

$$x[0] = x_{r,0} + jx_{i,0}, \tag{3.3a}$$

$$x[1] = x_{r,1} + jx_{i,1}, \tag{3.3b}$$

where $x_{r,0} = \Re\{x[0]\}$ and $x_{i,0} = \Im\{x[0]\}$ ($\Re\{\gamma\}$ and $\Im\{\gamma\}$ refer to the real and imaginary parts of γ, respectively). Equations (2.4a) and (2.4b) can be rewritten as

$$X[0] = X_{r,0} + jX_{i,0} = x_{r,0} + x_{i,1} + j(x_{i,0} + x_{i,1}), \tag{3.4a}$$

$$X[1] = X_{r,1} + jX_{i,1} = x_{r,0} - x_{i,1} + j(x_{i,0} - x_{i,1}). \tag{3.4b}$$

In this case, it is not necessary to operate real and imaginary parts together due to the fact that twiddle factors are real values. The minimum number of real adders that needs a circuit for implement these equations is 2. It is an easy task to use only 2 real adders and of 2 serial-parallel permutation circuits for operating on real and imaginary parts that arrives in consecutive clock cycles.

Figure 3.3 represents the proposed radix-2 serial butterfly circuit with minimum number of elements using only 2 real adders that are equivalent to 1 complex adder. The outer parts of the circuit are formed by serial-parallel permutation circuits. Table 3.1 shows its timing diagram for every signal that appears in the circuit, including control signals (S_0 and S_1).

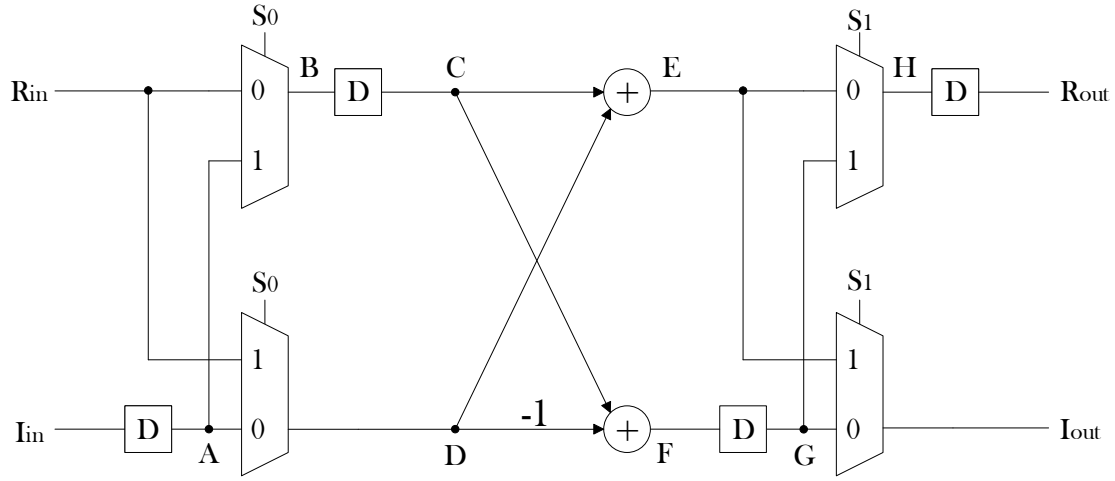

Figure 3.3: Proposed radix-2 serial butterfly

Signal	Time			
Clk cycle	0	1	2	3
S_0	0	1	0	1
S_1	0	0	1	0
R_{in}	$x_{r,0}$	$x_{r,1}$		
I_{in}	$x_{i,0}$	$x_{i,1}$		
A		$x_{i,0}$	$x_{i,1}$	
B	$x_{r,0}$	$x_{i,0}$		
C		$x_{r,0}$	$x_{i,0}$	
D		$x_{r,1}$	$x_{i,1}$	
E		$X_{r,0}$	$X_{i,0}$	
F		$X_{r,1}$	$X_{i,1}$	
G			$X_{r,1}$	$X_{i,1}$
H		X_r	Y_r	
R_{out}			$X_{r,0}$	$X_{r,1}$
I_{out}			$X_{i,0}$	$X_{i,1}$

Table 3.1: Timing diagram of the proposed radix-2 serial butterfly in Figure 3.3.

The proposed serial implementation of Figure 3.3 is going to be compared with a parallel one, to show how many resources are saved in the serial one. The parallel implementation of a radix-2 butterfly is represented in Figure 3.4 with real and imaginary parts split.

Table 3.2 shows the number of hardware resources used in both serial and parallel architectures. The parallel circuit that appears in this table does not need registers for keeping its functionality as are shown in Figure 3.4. By using the serial implementation, the system saves two real adders for the butterfly, with the additional cost of four multiplexers and four registers.

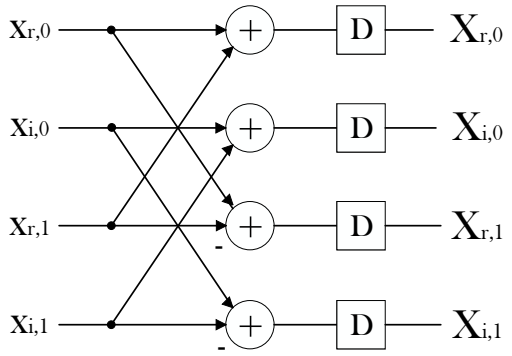

Figure 3.4: Proposed radix-2 parallel butterfly

Elements	Serial pipelined circuit	Parallel circuit
Multiplexers	4	0
Registers	4	0
Real adders	2	4

Table 3.2: Comparison between the proposed serial radix-2 butterfly and a parallel radix-2 butterfly

In order to obtain experimental results, it has been considered that input and output signals have a word length $WL = 8$ bits. In order to increase the maximum operating clock frequency, registers have been added at the end of each branch in both serial and parallel circuits, as shows Figure 3.5 and they are taken into account in Table 3.3. In Table 3.4, it has been proved with a word length $WL = 16$ bits.

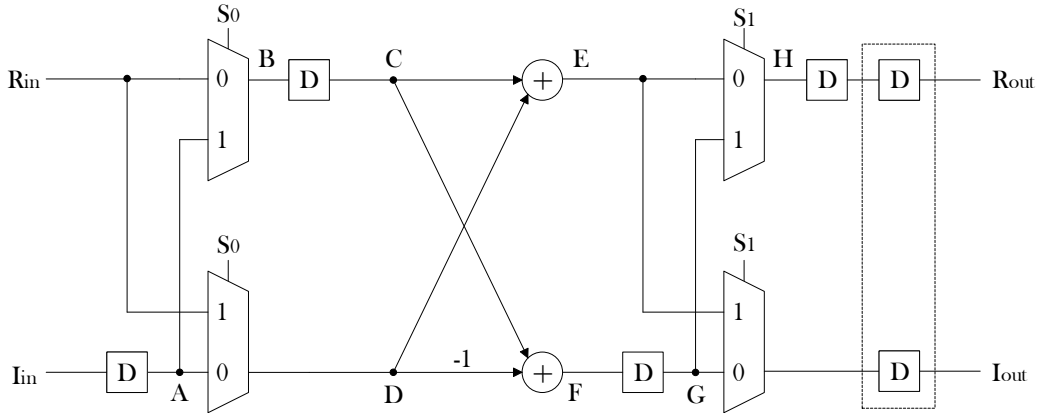

Figure 3.5: Proposed radix-2 serial butterfly for increasing clock frequency

Implementation elements in FPGA	Serial pipelined circuit	Parallel circuit
LUTs	38	41
Registers	51	33
Slices	15	15
Maximum operating frequency	261 MHz	349 MHz

Table 3.3: Comparison of the proposed radix-2 serial butterfly and the radix-2 parallel butterfly for $WL = 8$ bits on a xc7a100T-1CSG324C FPGA

Implementation elements in FPGA	Serial pipelined circuit	Parallel circuit
LUTs	74	81
Registers	99	65
Slices	25	27
Maximum operating frequency	229 MHz	348 MHz

Table 3.4: Comparison of the proposed radix-2 serial butterfly and the radix-2 parallel butterfly for $WL = 16$ bits on a xc7a100T-1CSG324C FPGA.

For $WL = 8$ and for $WL = 16$, the number of LUTs are slightly reduced in the serial implementation compared to the parallel one. For $WL = 8$ there is no difference in the number of slices, whereas for $WL = 16$ the number of slices is reduced slightly. Finally, both architectures achieve a high clock frequency, being higher for the parallel case.

3.3 Radix-3 butterfly implementation

Equations (2.5b), (2.5c) and (2.5c) can be rewritten as

$$X[0] = x_{r,0} + x_{r,1} + x_{r,2} + j(x_{i,0} + x_{i,1} + x_{i,2}), \tag{3.5a}$$

$$X[1] = x_{r,0} - \frac{1}{2}(x_{r,1}+x_{r,2}) + \frac{\sqrt{3}}{2}(x_{i,1}-x_{i,2}) + j\left(x_{i,0} - \frac{1}{2}(x_{i,1}+x_{i,2}) - \frac{\sqrt{3}}{2}(x_{r,1}-x_{r,2})\right), \tag{3.5b}$$

$$X[2] = x_{r,0} - \frac{1}{2}(x_{r,1}+x_{r,2}) - \frac{\sqrt{3}}{2}(x_{i,1}-x_{i,2}) + j\left(x_{i,0} - \frac{1}{2}(x_{i,1}+x_{i,2}) + \frac{\sqrt{3}}{2}(x_{r,1}-x_{r,2})\right). \tag{3.5c}$$

In this case, it is necessary to operate the real and imaginary parts together due to the fact that some twiddle factors have complex values. The minimum number of

real adders that a circuit needs for implementing these equations is 4. The proposed circuit has been designed with 6 real adders due to the fact that it makes the circuit more regular. The proposed circuit is composed of three stages. α, β, γ, δ, ϵ and ζ are the intermediate calculations of the butterfly. Table 3.5 includes the operations of each variable all along the stages so it is possible to see how operations are calculated and the utilization of the adders.

First stage			
α	β	γ	δ
$x_{r,1} + x_{r,2}$	$x_{r,1} - x_{r,2}$	$x_{i,1} + x_{i,2}$	$x_{i,1} - x_{i,2}$
Second stage			
ϵ	ζ	$X_{r,0}$	$X_{i,0}$
$x_{r,0} - \frac{\alpha}{2}$	$x_{i,0} - \frac{\gamma}{2}$	$x_{r,0} + \alpha$	$x_{i,0} + \gamma$
Third stage			
$X_{r,1}$	$X_{i,1}$	$X_{r,2}$	$X_{i,2}$
$\epsilon + \frac{\sqrt{3}}{2}\delta$	$\zeta - \frac{\sqrt{3}}{2}\beta$	$\epsilon - \frac{\sqrt{3}}{2}\delta$	$\zeta + \frac{\sqrt{3}}{2}\beta$

Table 3.5: Calculation stages in a radix-3 unit.

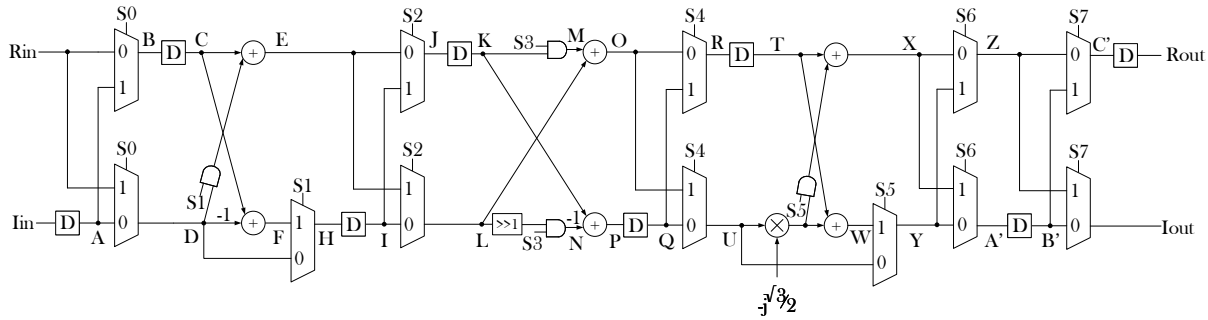

Figure 3.6: Proposed radix-3 serial butterfly.

Figure 3.6 represents the proposed radix-3 serial butterfly. It uses 6 real adders and only one multiplier. The design has saved a multiplier due to the fact that in the proposed architecture the multiplier is reused for the real and imaginary parts of the data. Table 3.6 shows the timing diagram for every signal that appears in the circuit, including the control signals.

Signal	Time						
Clk cycle	0	1	2	3	4	5	6
S_0	0	0	1	0	0	1	0
S_1	0	0	1	1	0	1	1
S_2	0	0	1	1	0	1	1
S_3	0	0	1	1	0	1	1
S_4	0	0	0	1	1	0	1
S_5	0	0	0	0	1	1	0
S_6	0	0	0	0	0	1	0
S_7	0	0	0	0	0	1	0
R_{in}	$x_{r,0}$	$x_{r,1}$	$x_{r,2}$				
I_{in}	$x_{i,0}$	$x_{i,1}$	$x_{i,2}$				
A		$x_{i,0}$	$x_{i,2}$	$x_{i,1}$			
B	$x_{r,0}$	$x_{r,1}$	$x_{i,1}$				
C		$x_{r,0}$	$x_{r,1}$	$x_{i,1}$			
D		$x_{i,0}$	$x_{r,2}$	$x_{i,2}$			
E		0	$x_{r,2}$	$x_{i,2}$			
F		-	β	δ			
G		$x_{r,0}$	α	γ			
H		$x_{i,0}$	β	δ			
I			$x_{i,0}$	β	δ		
J		$x_{r,0}$	$x_{i,0}$	β			
K			$x_{r,0}$	$x_{i,0}$	β		
L			α	γ	δ		
M			$x_{r,0}$	$x_{i,0}$	0		
N			$\frac{\alpha}{2}$	$\frac{\gamma}{2}$	0		
O			$X_{r,0}$	$X_{i,0}$	δ		
P			ϵ	ζ	β		
Q				ϵ	ζ	β	
R			$X_{r,0}$	ϵ	ζ		
T				$X_{r,0}$	ϵ	ζ	
U				$X_{i,0}$	δ	β	
V				0	$\frac{\sqrt{3}}{2}\delta$	$\frac{\sqrt{3}}{2}\beta$	
W				-	$X_{r,2}$	$X_{i,1}$	
X				$X_{r,0}$	$X_{r,1}$	$X_{i,2}$	
Y				$X_{i,0}$	$X_{r,2}$	$X_{i,1}$	
Z				$X_{r,0}$	$X_{r,1}$	$X_{i,1}$	
A'				$X_{i,0}$	$X_{r,2}$	$X_{i,2}$	
B'					$X_{i,0}$	$X_{r,2}$	$X_{i,2}$
C'				$X_{r,0}$	$X_{r,1}$	$X_{r,2}$	
R_{out}					$X_{r,0}$	$X_{r,1}$	$X_{r,2}$
I_{out}					$X_{i,0}$	$X_{i,1}$	$X_{i,2}$

Table 3.6: Timing diagram of the proposed radix-3 serial butterfly in Figure 3.6.

Figure 3.7 shows the parallel circuit used to calculate the radix-3 butterfly. Table

3.7 contains the difference in number of elements between both versions (by assuming that the parallel butterfly does not need any register to work).

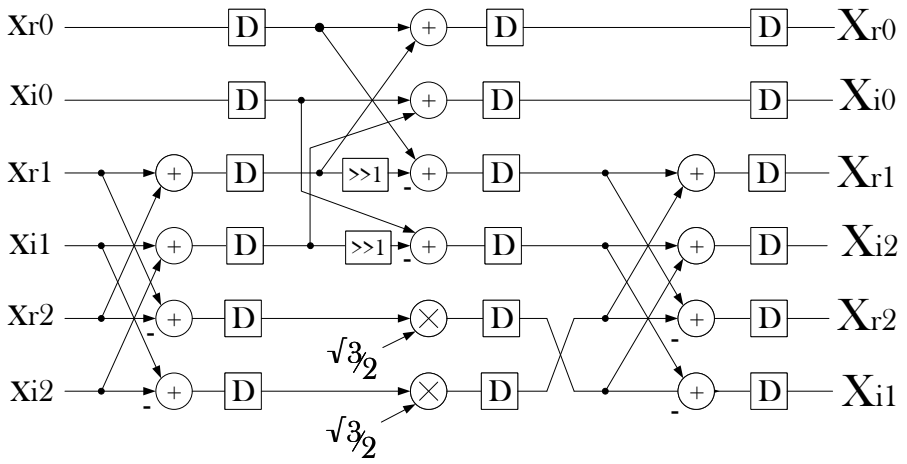

Figure 3.7: Radix-3 parallel circuit.

Elements	Serial pipelined circuit	Parallel circuit
Multiplexers	12	0
Registers	8	0
Real multipliers	1	2
Logic gates	4	0
Real adders	6	8

Table 3.7: Comparison between the proposed serial radix-3 butterfly and a parallel radix-3 butterfly

Tables 3.8 shows the resulting elements for the implementation of radix-3 serial butterfly for $WL = 8$ bits and for $WL = 16$ bits. No growth in word length after adders is considered.

Implementation elements in FPGA, WL	8	16
LUTs	178	255
Registers	67	131
Slices	62	83
Maximum operating frequency	77 MHz	66 MHz

Table 3.8: Implementation results of the proposed radix-3 serial butterfly for $WL = 8$ bits and for $WL = 16$ bits on a xc7a100T-1CSG324C FPGA.

The serial circuit was not designed for reach high frequencies. To increase the maximum operating frequency another design must be proposed including more registers after each stage.

3.4 Radix-4 butterfly implementation

Equations (2.7b), (2.7c), (2.7c) and (2.7d) can be rewritten as

$$X[0] = x_{r,0} + x_{r,1} + x_{r,2} + x_{r,3} + j(x_{i,0} + x_{i,1} + x_{i,2} + x_{i,3}), \tag{3.6a}$$

$$X[2] = x_{r,0} + x_{r,2} - x_{r,1} - x_{r,3} + j(x_{i,0} + x_{i,2} - x_{i,1} - x_{i,3}), \tag{3.6b}$$

$$X[1] = x_{r,0} - x_{r,2} + x_{i,1} - x_{i,3} + j(x_{i,0} - x_{i,2} - x_{r,1} + x_{r,3}), \tag{3.6c}$$

$$X[3] = x_{r,0} - x_{r,2} - x_{i,1} + x_{i,3} + j(x_{i,0} - x_{i,2} + x_{r,1} - x_{r,3}). \tag{3.6d}$$

Table 3.9 includes the operations of each variable and it represents how operations have been distributed among both stages. α, β, γ, δ, ϵ, ζ, η and θ are the intermediate variables.

First stage							
α	β	γ	δ	ϵ	ζ	η	θ
$x_{r,0}$ + $x_{r,2}$	$x_{r,1}$ + $x_{r,3}$	$x_{i,0}$ + $x_{i,2}$	$x_{i,1}$ + $x_{i,3}$	$x_{r,0}$ - $x_{r,2}$	$x_{i,1}$ - $x_{i,3}$	$x_{i,0}$ - $x_{i,2}$	$x_{r,1}$ - $x_{r,3}$
Second stage							
$X_{r,0}$	$X_{i,0}$	$X_{r,2}$	$X_{i,2}$	$X_{r,1}$	$X_{i,1}$	$X_{r,3}$	$X_{i,3}$
α+β	γ+δ	α-β	γ-δ	ϵ+ζ	η-θ	ϵ-ζ	η+θ

Table 3.9: Calculation stages in a radix-4 unit.

The minimum number of real adders that a circuit needs for implement equations (3.6) is 4. The number of real adders used in the proposed circuit is also 4. The magnitude of the twiddle factor is one.

Some designs of radix-4 butterflies are proposed in this section due to the fact that different scenarios have been considered. In the first one, Figure 3.8, it is supposed that data arrive in bit-reversed (BR) order and the outputs are provided in bit-reversed order too (BR-BR). This allows to simplify the circuit. In the other three implementations, data is received or provided in different orders. Figure 3.9 includes additional hardware that changes the output order from BR to natural order (BR-N). Figure 3.10 is the symmetric structure of the BR-N (N-BR), so additional hardware is included at the output branches. The last one, Figure 3.11, represents the butterfly when both input and output data are in natural order. Then, a parallel circuit which calculates a radix-4 butterfly is shown in Figure 3.12 in order to compare it with serial ones.

Table 3.10 shows the timing diagram for the circuit in Figure 3.8. Every signal that appears in the circuit, including control signals, is included in the timing diagram.

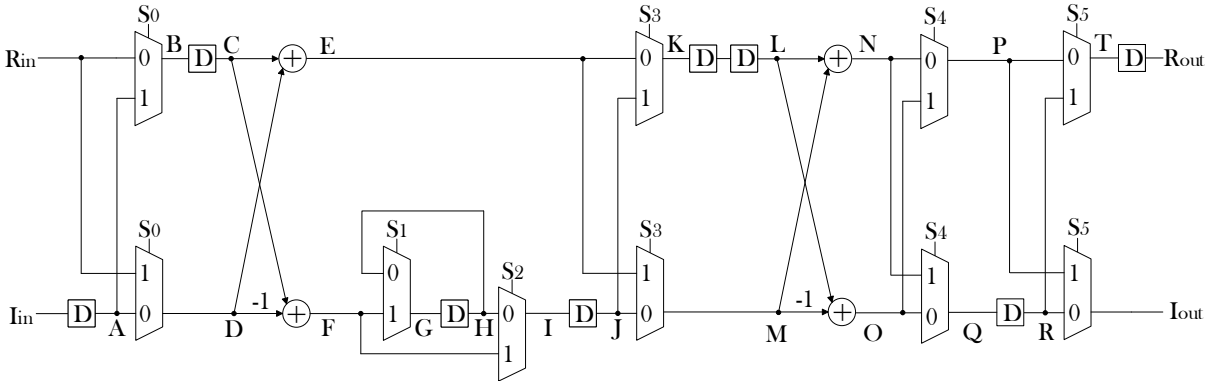

Figure 3.8: Proposed radix-4 serial butterfly with I/O order BR-BR.

Signal	Time							
Clk cycle	0	1	2	3	4	5	6	7
S_0	0	1	0	1	0	1	0	1
S_1	0	1	1	1	0	1	1	1
S_2	0	0	0	0	1	0	0	0
S_3	0	0	0	1	1	0	0	1
S_4	0	0	0	0	0	0	1	0
S_5	0	0	0	0	1	0	1	0
R_{in}	$x_{r,0}$	$x_{r,2}$	$x_{r,1}$	$x_{r,3}$				
I_{in}	$x_{i,0}$	$x_{i,2}$	$x_{i,1}$	$x_{i,3}$				
A		$x_{i,0}$	$x_{i,2}$	$x_{i,1}$	$x_{i,3}$			
B	$x_{r,0}$	$x_{i,0}$	$x_{r,1}$	$x_{i,1}$				
C		$x_{r,0}$	$x_{i,0}$	$x_{r,1}$	$x_{i,1}$			
D		$x_{r,2}$	$x_{i,2}$	$x_{r,3}$	$x_{i,1}$			
E		α	γ	β	δ			
F		ϵ	η	θ	ζ			
G		ϵ	η	θ	θ			
H			ϵ	η	θ	θ		
I			ϵ	η	ζ	θ		
J				ϵ	η	ζ	θ	
K		α	γ	ϵ	η			
L				α	γ	ϵ	η	
M				β	δ	ζ	θ	
N				$X_{r,0}$	$X_{i,0}$	$X_{r,1}$	$X_{i,3}$	
O				$X_{r,2}$	$X_{i,2}$	$X_{r,3}$	$X_{i,1}$	
P				$X_{r,0}$	$X_{i,0}$	$X_{r,1}$	$X_{i,1}$	
Q				$X_{r,2}$	$X_{i,2}$	$X_{r,3}$	$X_{i,3}$	
R					$X_{r,2}$	$X_{i,2}$	$X_{r,3}$	$X_{i,3}$
T				$X_{r,0}$	$X_{r,2}$	$X_{r,1}$	$X_{r,3}$	
R_{out}					$X_{r,0}$	$X_{r,2}$	$X_{r,1}$	$X_{r,3}$
I_{out}					$X_{i,0}$	$X_{i,2}$	$X_{i,1}$	$X_{i,3}$

Table 3.10: Timing diagram of the proposed radix-4 BR-BR butterfly.

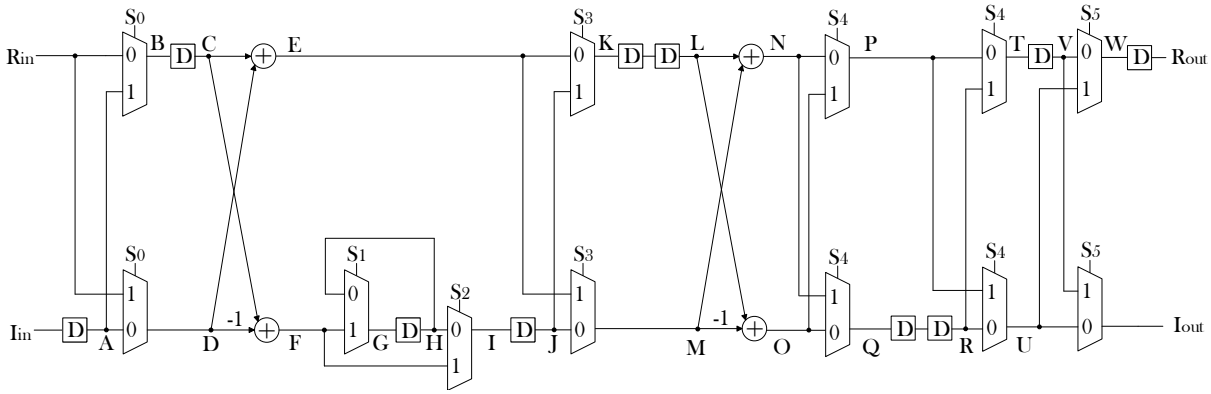

Figure 3.9: Proposed radix-4 serial butterfly with I/O order BR-N.

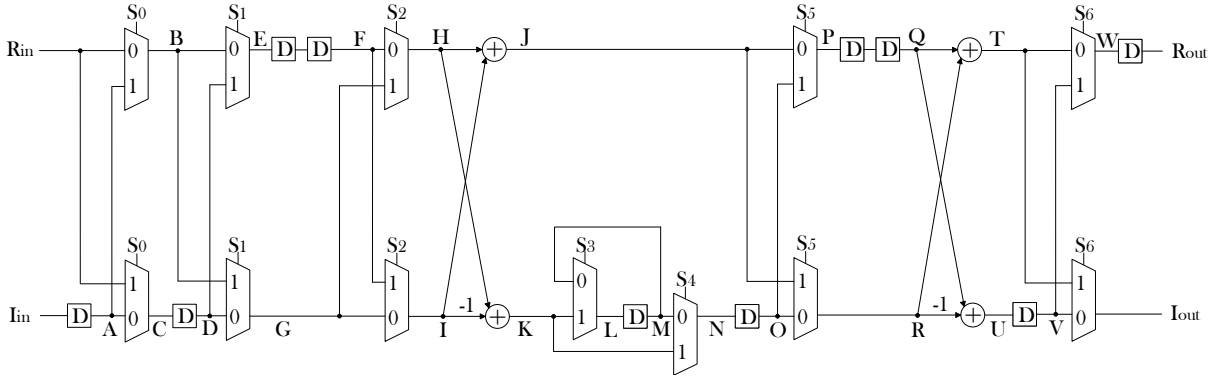

Figure 3.10: Proposed radix-4 serial butterfly with I/O order N-BR.

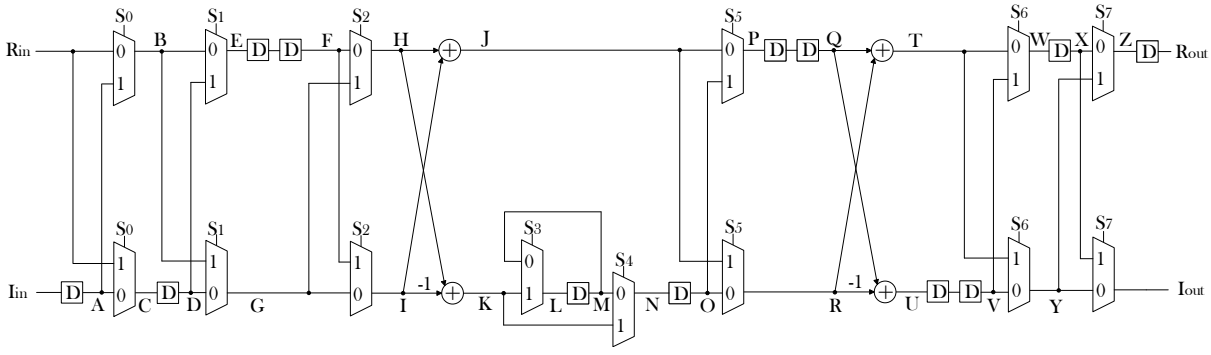

Figure 3.11: Proposed radix-4 serial butterfly with I/O order N-N.

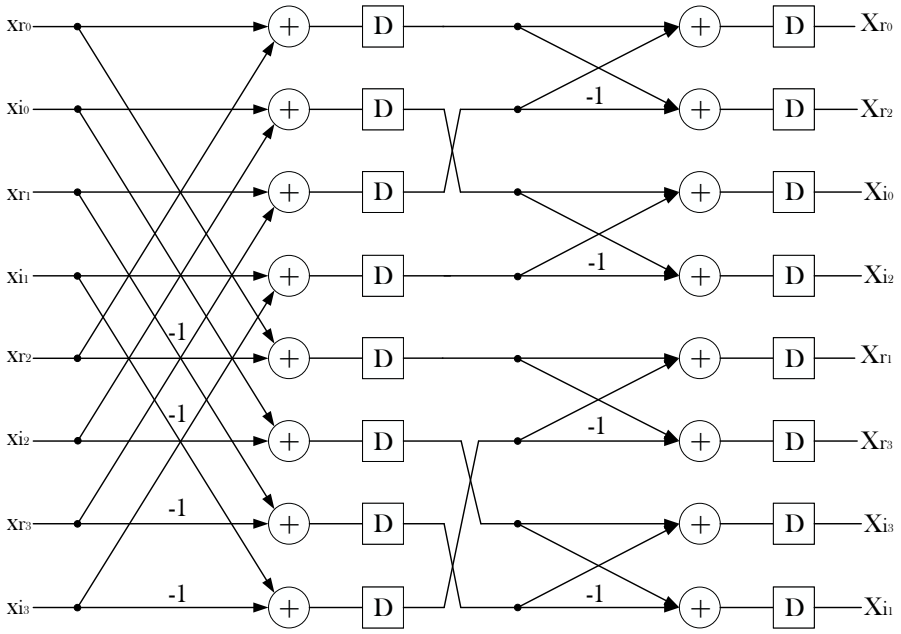

Figure 3.12: Radix-4 Parallel Circuit.

Table 3.11 shows the difference between the number of hardware resources used in both serial and parallel architectures. For the serial implementation, the BR-BR case is considered (Figure 3.8). The proposed architecture saves 12 real adders with the cost of 10 additional multiplexers and 8 registers.

Elements	Serial pipelined circuit	Parallel circuit
Multiplexers	10	0
Registers	8	0
Real adders	4	16

Table 3.11: Comparison between the proposed serial radix-4 butterfly and a parallel radix-4 butterfly.

Table 3.12 shows the experimental results for $WL = 8$.

Implementation elements in FPGA	Serial pipelined circuit	Parallel circuit
LUTs	90	146
Registers	68	130
Slices	33	56
Maximun operating frequency	164 MHz	373 MHz

Table 3.12: Comparison of the proposed radix-4 serial butterfly and the radix-4 parallel butterfly for $WL = 8$ bits on a xc7a100T-1CSG324C FPGA.

The proposed serial radix-4 butterfly uses less hardware elements, but also has a lower frequency due to the fact that the combinational time is larger. In order to solve this issue, an additional circuit has been implemented, Figure 3.13. Registers are included in both branches immediately after the adders and at the end of the circuit, which increases the maximum operating frequency. Tables 3.13 and 3.14 compare the new circuit and the parallel version (not changed) for a word length of 8 and 16 bits, respectively.

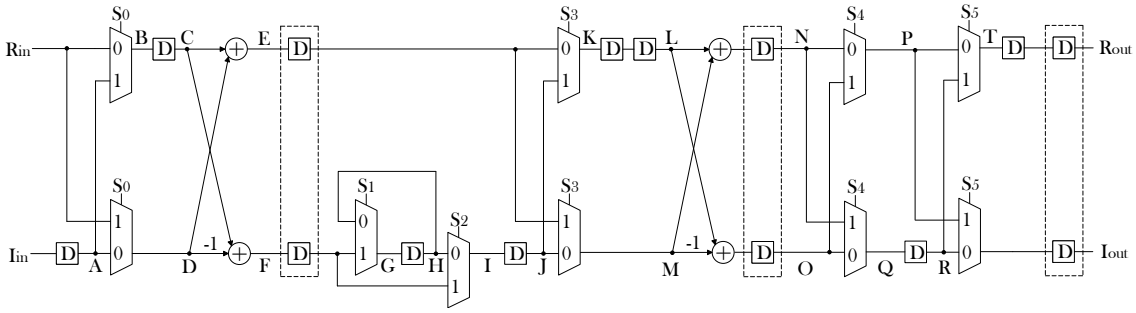

Figure 3.13: Proposed radix-4 butterfly for increasing clock frequency.

Implementation elements in FPGA	Serial pipelined circuit	Parallel circuit
LUTs	90	146
Registers	117	130
Slices	34	56
Maximum operating frequency	278 MHz	373 MHz

Table 3.13: Comparison of the proposed radix-4 serial butterfly and the radix-4 parallel butterfly for $WL = 8$ bits on a xc7a100T-1CSG324C FPGA.

Implementation elements in FPGA	Serial pipelined circuit	Parallel circuit
LUTs	170	290
Registers	229	258
Slices	60	92
Maximum operating frequency	261 MHz	307 MHz

Table 3.14: Comparison of the proposed radix-4 serial butterfly and the radix-4 parallel butterfly for $WL = 16$ bits on a xc7a100T-1CSG324C FPGA.

The number of LUTs and slices are significantly reduced in the proposed serial butterfly compared to the parallel implementation. The effect of using only two real adders makes the difference. Finally, both architectures achieve a high clock frequency, being higher for the parallel case.

3.5 Radix-5 butterfly implementation

Equations (2.10a), (2.10b), (2.10c), (2.10d) and (2.10e) can be rewritten as

$$X[0] = x_{r,0} + x_{r,1} + x_{r,2} + x_{r,3} + x_{r,4} + j(x_{i,0} + x_{i,1} + x_{i,2} + x_{i,3} + x_{i,4}), \tag{3.7a}$$

$$\begin{aligned} X[1] = {} & x_{r,0} + K_1(x_{r,1} + x_{r,2} + x_{r,3} + x_{r,4}) + K_2((x_{r,1} + x_{r,4}) - (x_{r,2} + x_{r,3})) \\ & - K_3(x_{i,1} - x_{i,4}) - K_4((x_{i,1} - x_{i,4}) + (x_{i,2} - x_{i,3})) \\ & + j(x_{i,0} + K_1(x_{i,1} + x_{i,2} + x_{i,3} + x_{i,4}) + K_2((x_{i,1} + x_{i,4}) - (x_{i,2} + x_{i,3})) \\ & + K_3(x_{r,1} - x_{r,4}) + K_4((x_{r,1} - x_{r,4}) + (x_{r,2} - x_{r,3}))), \end{aligned} \tag{3.7b}$$

$$\begin{aligned} X[2] = {} & x_{r,0} + K_1(x_{r,1} + x_{r,2} + x_{r,3} + x_{r,4}) - K_2((x_{r,1} + x_{r,4}) - (x_{r,2} + x_{r,3})) \\ & - K_5(x_{i,2} - x_{i,3}) - K_4((x_{i,1} - x_{i,4}) + (x_{i,2} - x_{i,3})) \\ & + j(x_{i,0} + K_1(x_{i,1} + x_{i,2} + x_{i,3} + x_{i,4}) - K_2((x_{i,1} + x_{i,4}) - (x_{i,2} + x_{i,3})) \\ & + K_5(x_{r,2} - x_{r,3}) + K_4((x_{r,1} - x_{r,4}) + (x_{r,2} - x_{r,3}))), \end{aligned} \tag{3.7c}$$

$$\begin{aligned} X[3] = {} & x_{r,0} + K_1(x_{r,1} + x_{r,2} + x_{r,3} + x_{r,4}) - K_2((x_{r,1} + x_{r,4}) - (x_{r,2} + x_{r,3})) \\ & + K_5(x_{i,2} - x_{i,3}) + K_4((x_{i,1} - x_{i,4}) + (x_{i,2} - x_{i,3})) \\ & + j(x_{i,0} + K_1(x_{i,1} + x_{i,2} + x_{i,3} + x_{i,4}) - K_2((x_{i,1} + x_{i,4}) - (x_{i,2} + x_{i,3})) \\ & - K_5(x_{r,2} - x_{r,3}) - K_4((x_{r,1} - x_{r,4}) + (x_{r,2} - x_{r,3}))), \end{aligned} \tag{3.7d}$$

$$\begin{aligned} X[4] = {} & x_{r,0} + K_1(x_{r,1} + x_{r,2} + x_{r,3} + x_{r,4}) + K_2((x_{r,1} + x_{r,4}) - (x_{r,2} + x_{r,3})) \\ & - K_3(x_{i,1} - x_{i,4}) + K_4((x_{i,1} - x_{i,4}) + (x_{i,2} - x_{i,3})) \\ & + j(x_{i,0} + K_1(x_{i,1} + x_{i,2} + x_{i,3} + x_{i,4}) + K_2((x_{i,1} + x_{i,4}) - (x_{i,2} + x_{i,3})) \\ & - K_3(x_{r,1} - x_{r,4}) - K_4((x_{r,1} - x_{r,4}) + (x_{r,2} - x_{r,3}))). \end{aligned} \tag{3.7e}$$

Table 3.15 includes the operations of each variable and it represents how operations have been distributed among each stage. The constants K_1, K_2, K_3, K_4 and K_5 that appear in the table are not the ones that are used in the flow graph due to the fact that those constants are complex-valued. Now the values of their real or imaginary part are used because the operands must be real in a real adder. These new constant values are given in the set of equations (3.8).

$$K_1 = \frac{1}{2}\left(\cos(\frac{2\pi}{5}) + \cos(\frac{4\pi}{5})\right) = -\frac{1}{4} \tag{3.8a}$$

$$K_2 = \frac{1}{2}\left(\cos(\frac{2\pi}{5}) - \cos(\frac{4\pi}{5})\right) = 0.56 \tag{3.8b}$$

First stage							
α	β	γ	δ	ϵ	ζ	η	θ
$x_{r,1} + x_{r,4}$	$x_{r,2} + x_{r,3}$	$x_{i,1} + x_{i,4}$	$x_{i,2} + x_{i,3}$	$x_{r,1} - x_{r,4}$	$x_{r,2} - x_{r,3}$	$x_{i,1} - x_{i,4}$	$x_{i,2} - x_{i,3}$
Second stage							
α'	β'	γ'	δ'	ϵ'	ζ'		
$\alpha + \beta$	$\alpha - \beta$	$\gamma + \delta$	$\gamma - \delta$	$\epsilon + \zeta$	$\eta + \theta$		
Third stage							
λ	μ	$X_{r,0}$	$X_{i,0}$				
$x_{r,0} + K_1\alpha'$	$x_{i,0} + K_1\gamma'$	$x_{r,0}+\alpha'$	$x_{i,0}+\gamma'$				
Fourth stage							
α''	β''	γ''	δ''	ϵ''	ζ''	η''	θ''
$\lambda+K_2\beta'$	$K_4\zeta' + K_3\eta$	$\mu+K_2\delta'$	$K_4\epsilon' + K_3\epsilon$	$\lambda - K_2\beta'$	$K_4\zeta' + K_5\theta$	$\mu - K_2\delta'$	$K_4\epsilon' + K_5\zeta$
Fifth stage							
$x_{r,1}$	$x_{i,1}$	$x_{r,2}$	$x_{i,2}$	$x_{r,3}$	$x_{i,3}$	$x_{r,4}$	$x_{i,4}$
$\alpha''-\beta''$	$\gamma''+\delta''$	$\epsilon''-\zeta''$	$\eta''+\theta''$	$\epsilon''+\zeta''$	$\eta''-\theta''$	$\alpha''+\beta''$	$\gamma''-\delta''$

Table 3.15: Calculation stages in a radix-5 unit.

$$K_3 = \sin(\frac{4\pi}{5}) - \sin(\frac{2\pi}{5}) = -0.36 \tag{3.8c}$$

$$K_4 = -\sin(\frac{4\pi}{5}) = -0.59 \tag{3.8d}$$

$$K_5 = \sin(\frac{4\pi}{5}) + \sin(\frac{2\pi}{5}) = 1.54. \tag{3.8e}$$

The minimum number of real adders that are needed in a circuit for implementing equations (3.7) is 7. The number of real adders used in the proposed circuit is 10 for the same reason as for radix-3 implementation, i.e. it is is important to make the circuit more regular.

The minimum number of real multipliers for implement those equations is just one. The way in which operations are distributed among the stages allows to use only one multiplier in the proposed design.

Radix-5 butterflies are the largest ones in the 5G standard allows (2.2). As shown in Figure 2.7, 17 complex additions must be calculated along the flow graph. This makes the design of a pipelined circuit a more complicated task than for other radices, because there are some points at the flow graph where there are more than 5 branches. That is automatically traduced in more than two branches at some points in the hardware implementation.

Figure 3.14 shows the proposed radix-5 serial butterfly and Table 3.17 presents its timing diagram. It is not possible to include all signals appearing in the circuit, so the timing diagram only includes those that are more important such as inputs

and outputs of permutation circuits or adders. Table 3.16 shows all the signals that control every multiplexer.

Control signal	Time													
Clk cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13
S_0	0	0	1	0	1	0	0	1	0	1	0	0	1	0
S_1	0	0	1	1	1	1	0	1	1	1	1	0	1	1
S_2	0	0	0	0	1	1	0	0	0	1	1	0	0	0
S_3	0	0	0	0	1	1	0	0	0	1	1	0	0	0
S_4	0	0	0	0	1	1	1	1	0	1	1	1	1	0
S_5	0	0	0	0	1	1	1	1	0	1	1	1	1	0
S_6	0	0	0	0	1	0	0	0	0	1	0	0	0	0
S_7	0	0	0	0	0	0	0	0	0	1	0	0	0	0
S_8	0	0	0	0	0	0	1	1	0	0	0	1	1	0
S_9	0	0	0	0	1	1	0	0	0	1	1	0	0	0
S_{10}	0	0	0	0	0	0	0	0	1	0	0	0	0	1
S_{11}	0	0	0	0	0	0	0	0	0	1	0	0	0	0
S_{12}	0	0	0	0	0	0	1	1	1	1	0	1	1	1
S_{13}	0	0	0	0	0	0	0	0	0	1	0	0	0	0
S_{14}	0	0	0	0	0	0	0	0	0	0	1	0	0	0
S_{15}	0	0	0	0	0	0	0	0	0	1	1	0	0	0
S_{16}	0	0	0	0	0	0	0	0	0	1	1	1	1	0
S_{17}	0	0	0	0	0	0	0	0	0	1	0	1	0	0
S_{18}	0	0	0	0	0	0	0	0	0	0	1	0	1	0

Table 3.16: Control signals of the timing diagram of radix-5 butterfly of Figure 3.14.

Table 3.18 shows the difference between the number of hardware resources used in both serial and parallel architectures. The architectures used in the comparison are the serial implementation (Figure 3.14) and a parallel one that is supposed to have registers after every arithmetic operation such as parallel versions of every other radix-ρ butterfly. The proposed serial radix-5 butterfly saves 24 real adders and 8 real multipliers with an increase in the number of multiplexers and logic gates. By assuming that the multipliers are significantly larger than multiplexers, registers and logic gates, the proposed architecture reduces significantly the number of hardware resources.

Signal	Time													
Clk cycle	0	1	2	3	4	5	6	7	8	9	10	11	12	13
R_{in}	$x_{r,0}$	$x_{r,1}$	$x_{r,4}$	$x_{r,2}$	$x_{r,3}$									
I_{in}	$x_{i,0}$	$x_{i,1}$	$x_{i,4}$	$x_{i,2}$	$x_{i,3}$									
C		$x_{r,0}$	$x_{r,1}$	$x_{i,1}$	$x_{r,2}$	$x_{i,2}$								
D		$x_{i,0}$	$x_{r,4}$	$x_{i,4}$	$x_{r,3}$	$x_{i,3}$								
E		$x_{r,0}$	α	γ	β	δ								
F		$-x_{i,0}$	ϵ	η	ζ	θ								
I				$x_{r,0}$	α	γ	ϵ	η						
J				$-x_{i,0}$	β	δ	ζ	θ						
K				$x_{r,0}$	α'	γ'	ϵ'	ζ'						
L				$x_{i,0}$	β'	δ'	$-\zeta$	$-\theta$						
O					α'	γ'	ϵ'	ζ'	ϵ					
P					β'	δ'	$-\zeta$	$-\theta$	η					
T					$x_{r,0}$	$x_{i,0}$	-	-	-					
U					$K_1\alpha'$	$K_1\gamma'$	$K_4\epsilon'$	$K_4\zeta'$	$-K_3\epsilon$					
V					$K_2\beta'$	$K_2\delta'$	$K_5\zeta'$	$K_5\theta'$	$-K_3\eta$					
W					$X_{r,0}$	$X_{i,0}$	-	-	-					
X					λ	μ	$K_4\epsilon'$	$K_4\zeta'$	$-K_3\epsilon$					
B'							λ	μ	$K_4\epsilon'$	$K_4\zeta'$	-			
C'							$K_2\beta'$	$K_2\delta'$	$-K_3\epsilon$	$-K_3\eta$				
D'							α''	γ''	θ''	ζ''				
E'							ϵ''	η''	δ''	β''				
G'						$X_{r,0}$	α''	γ''	θ''	ζ''				
H'						$-X_{i,0}$	ϵ''	η''	δ''	β''				
Q'									$X_{r,0}$	α''	γ''	ϵ''	η''	
R'									$-X_{i,0}$	β''	δ''	ζ''	θ''	
T'									$X_{r,0}$	$X_{r,4}$	$X_{i,1}$	$X_{r,3}$	$X_{i,2}$	
U'									$X_{i,0}$	$X_{r,1}$	$X_{i,4}$	$X_{r,2}$	$X_{i,3}$	
R_{out}										$X_{r,0}$	$X_{r,1}$	$X_{r,4}$	$x_{r,2}$	$X_{r,3}$
I_{out}										$X_{i,0}$	$X_{i,1}$	$X_{i,4}$	$x_{i,2}$	$X_{i,3}$

Table 3.17: Timing diagram of the proposed radix-5 serial butterfly.

Elements	Serial pipelined circuit	Parallel circuit
Multiplexers	25	0
Registers	23	0
AND logic gates	7	0
Real multipliers	2	10
Real adders	10	34

Table 3.18: Comparison between the proposed serial radix-5 butterfly and a parallel radix-5 butterfly.

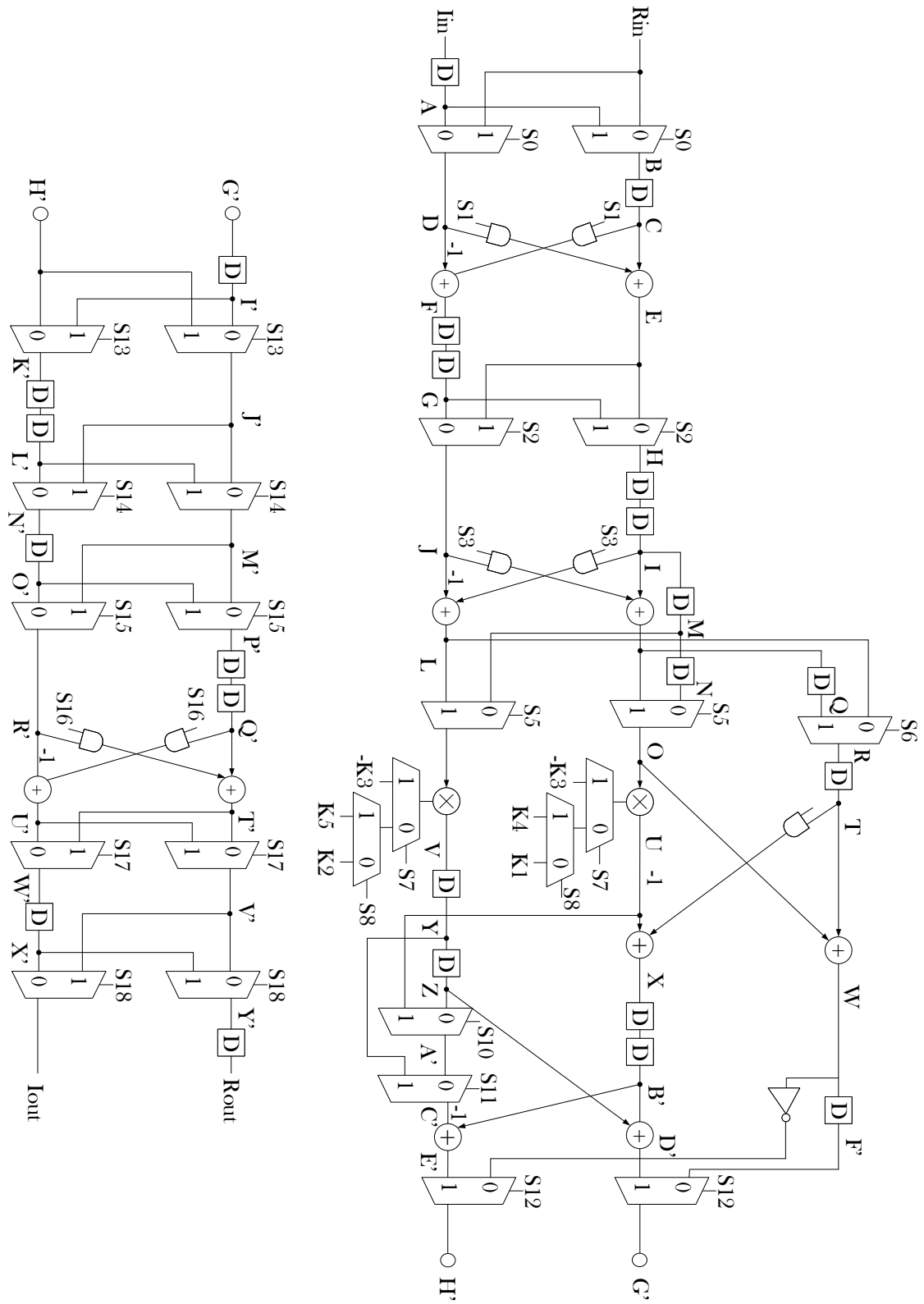

Figure 3.14: Proposed radix-5 serial butterfly.

Chapter 4

Conclusions and future work

4.1 Conclusions

This TFG has presented the hardware design of new serial butterflies of radix-ρ for $\rho = 2$, 3, 4 and 5, that process data in series. They consist of pipelined architectures that have a single input branch and process one datum per clock cycle in a continuous flow.

The proposed designs make use of permutation circuits that are proved to be optimal. Thus, a low number of registers and multiplexers have been used. The number of arithmetic elements, such as adders and multipliers, used for the proposed butterflies has been successfully reduced compared to parallel butterflies. In case of radix-2 and radix-4 butterflies the number of adders is the minimum possible, and the number of multipliers for radix-3 and radix-5 butterflies are also the minimum possible that a serial architecture can reach.

The proposed hardware designs (with the exception of radix-5 butterfly) have been implemented on an FPGA in order to prove their functionality. Parallel architectures have also been implemented and compared with the serial ones. The comparisons include the numbers of LUTs, registers and slices used in the implementation. The experimental results lead to concluding that proposed serial pipelined architectures consume a lower number of resources than some parallel ones. It exists a larger difference in the number of adders between radix-4 serial butterfly and radix-4 parallel butterfly than between radix-2 serial butterfly and radix-2 parallel butterfly, so that the bigger ρ, the larger number of resources saved.

4.2 Future work

The implementation of the proposed radix-5 butterfly design on the FPGA and the implementation of a new radix-3 butterfly design which increases its maximum operating frequency have been left for the future due to lack of time.

Future work concerns a higher grade of optimization of the proposed circuits in terms of hardware resources and latency. The circuits of radix-3 and radix-5 butterflies can be improved by using a lower number of adders but making them less regular. Every circuit presented in chapter 3 can increase its maximum operating frequency in order to adapt it to the latency expected for new communication technologies such as for 6G networks.

Regarding the range of radices, it could be interesting to implement other sizes for the butterflies, for example radix-7 ones. This would widen the range of alternatives for computing NP2 FFTs using serial architectures.

Appendix A

Budget

LABOUR COSTS		Hours	Cost/hour	TOTAL
Junior Telecommunication Engineer		300	15 €	**4.500 €**
MATERIAL COSTS	Cost (€)	Use (months)	Amort. (years)	TOTAL
Computer and sofware	1.500,00	6	5	**150,00 €**
FPGA	250,00	6	5	**25,00 €**
Laser printer	500,00	6	5	**50,00 €**
Other equipment				**0,00 €**
SUBTOTAL DIRECT COSTS				**4725,00 €**
INDIRECT COSTS	15%	of DC		**705,00 €**
INDUSTRIAL PROFIT	6%	of DC+IC		**324,30 €**
FUNGIBLE MATERIAL				
Printing				**100,00 €**
Binding				**300,00 €**
SUBTOTAL				**6.154,30 €**
IVA	21%			**1.292.40 €**
TOTAL BUDGET				**7.446,70 €**

Table A.1: Budget

Appendix B

Ethical, social, economic and environmental aspects

This annex discusses ethical, economic, social and environmental aspects related to this Bachelor's Thesis.

B.1 Ethical and social impacts

The greatest effect of the development of these new architectures will be seen when the 6G technology is implemented. This technology is expected to be implemented at the end of this decade, providing its benefits to the high majority of the world population.

B.2 Economic impact

Since TIC business entered the market during the last century, every single advance that concerns saving resources for a same application means an advantage for companies. The experimental results of this TFG demonstrates that it is possible to compute the algorithm of FFT with lower resources. Consequently, companies will benefit from incorporating the advances proposed in this TFG.

B.3 Environmental impact

The proposed designs will improve the energy consumption and reduce the area of FFT architectures used in 5G and beyond. This will result in more sustainable and environmentally friendly devices.

Bibliography

[1] 3GPP. Physical channels and modulation. *TS*, V16.3.0:Rel. 16, September 2020.

[2] Mario Garrido and Pedro Malagón. The constant multiplier fft. *IEEE Trans. Circuits Syst. I*, Vol. 68, No. 1:322–325, January 2021.

[3] S.-G. Chen W.-L. Tsai and S.-J. Huang. Reconfigurable radix-2k x 3 feedforward fft architectures. *IEEE Int. Symp. Circuits Syst.*, May 2019.

[4] H.-R. Chou X.-Y. Shih and Y.-Q. Liu. Vlsi design and implementation of reconfigurable 46-mode combined-radix-based FFT hardware architecture for 3gpp-lte applications. *IEEE Trans. Circuits Syst. I*, 65:118–129, January 2018.

[5] J. W. Cooley and J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Math. Comput.*, 19(90):297–301, April 1965.

[6] Volker Springel. The cosmological simulation code gadget-2. *Monthly Notices of the Royal Astronomical Society*, 364(4):1105–1134, 2005.

[7] S.R.J. Axelsson. Improved fourier modeling of soil temperature using fft algorithms. *IEEE Transactions on Geoscience and Remote Sensing*, 36(5):1519–1523, 1998.

[8] Nandana Rajatheva, Italo Atzeni, Emil Björnson, André Bourdoux, Stefano Buzzi, Jean-Baptiste Doré, Serhat Erkucuk, Manuel Fuentes, Ke Guan, Yuzhou Hu, Xiaojing Huang, Jari Hulkkonen, Josep Miquel Jornet, Marcos Katz, Rickard Nilsson, Erdal Panayirci, Khaled Rabie, Nuwanthika Rajapaksha, Mohammad Javad Salehi, Hadi Sarieddeen, Shahriar Shahabuddin, Tommy Svensson, Oskari Tervo, Antti Tölli, Qingqing Wu, and Wen Xu. White paper on broadband connectivity in 6G. *6G Research Vision*, (10):1–48, June 2020.

[9] A.V. Oppenheim and R.W. Schafer. *Discrete-Time Signal Processing*. Prentice Hall, 1989.

[10] M. Garrido, J. Grajal, and O. Gustafsson. Optimum circuits for bit reversal. *IEEE Trans. Circuits Syst. II*, 58(10):657–661, October 2011.

[11] J. Löfgren and P. Nilsson. On hardware implementation of radix 3 and radix 5 fft kernels for lte systems. In *2011 NORCHIP*, pages 1–4, 2011.

[12] S. Winograd. On computing the discrete Fourier transform. *Math. Comput.*, 32(141):175–199, January 1978.

[13] C. M. Rader. Discrete Fourier transform when the number of data samples is prime. *Proc. IEEE*, 56(6):1107–1108, June 1968.

[14] M. Garrido, J. Grajal, and O. Gustafsson. Optimum circuits for bit-dimension permutations. *IEEE Trans. VLSI Syst.*, 27(5):1148–1160, May 2019.

[15] N. Bhagat, D. Valencia, A. Alimohammad, and F. Harris. High-throughput and compact FFT architectures using the Good–Thomas and Winograd algorithms. *IET Commun.*, 12(8):1011–1018, May 2018.

[16] Mario Garrido, Rikard Andersson, Fahad Qureshi, and Oscar Gustafsson. Multiplierless unity-gain SDF FFTs. *IEEE Trans. VLSI Syst.*, 24(9):3003–3007, September 2016.

[17] M. Bansal and S. Nakhate. High speed pipelined 64-point FFT processor based on radix-2^2 for wireless LAN. In *Proc. Int. Conf. Signal Process. Integrated Networks*, pages 607–612, February 2017.

[18] V. M. Milovanović and M. L. Petrović. A highly parametrizable chisel HCL generator of single-path delay feedback FFT processors. In *Proc. IEEE Int. Conf. Microelectron.*, pages 247–250, September 2019.

[19] Ainhoa Cortes, Igone Velez, and Juan F. Sevillano. Radix r^k ffts: Matricial representation and sdc/sdf pipeline implementation. *IEEE Transactions on Signal Processing*, 57(7):2824–2839, July 2009.

[20] I. Cho, T. Patyk, D. Guevorkian, J. Takala, and S. Bhattacharyya. Pipelined FFT for wireless communications supporting 128–2048/1536-point transforms. In *Proc. IEEE Global Conf. Signal Inf. Process.*, pages 1242–1245, December 2013.

[21] Jinkyu Kim, Joohyun Lee, and Kyoungrok Cho. A mixed-radix pipeline FFT processor with trivial multiplications for LTE uplink. In *Proc. IEEE Int. Symp. Consumer Electron.*, pages 57–58, September 2016.

[22] Mario Garrido, Shen-Jui Huang, Sau-Gee Chen, and Oscar Gustafsson. The serial commutator (SC) FFT. *IEEE Trans. Circuits Syst. II*, 63(10):974–978, October 2016.

[23] M. Garrido. A survey on pipelined FFT hardware architectures. *J. Signal Process. Syst.*, 2021. Accepted for publication.

[24] J. Löfgren, L. Liu, O. Edfors, and P. Nilsson. Improved matching-pursuit implementation for LTE channel estimation. *IEEE Trans. Circuits Syst. I*, 61(1):226–237, January 2014.

[25] X.-Y. Shih, H.-R. Chou, and Y.-Q. Liu. VLSI design and implementation of reconfigurable 46-mode combined-radix-based FFT hardware architecture for 3GPP-LTE applications. *IEEE Trans. Circuits Syst. I*, 65(1):118–129, January 2018.